Assignment: Building Resilient Distributed Systems

Ifra | BSCS 23074

Github link: https://github.com/ifra-r/PDC-Sp24-bscs23074-Rauf

Part 1: Analysis of the Failure Points

StudySync's MVP is a textbook example of a system built for the happy path. Each of the three bugs stems from ignoring what happens when two things happen at the same time, when a message gets dropped, or when a dependency stops responding. Here is the root cause of each.

Problem 1: The Lost Update (Synchronization)

The root cause is a total absence of concurrency control at the database layer. The update flow is a straightforward read-modify-write: the client fetches the document, makes changes, and sends a PUT request that unconditionally overwrites the stored record. There is no version check, no lock, and no comparison against what was there before the edit.

When two users open the same document simultaneously, they both receive version N. User A submits their changes first and the record becomes version N+1. User B then submits their changes, but their PUT still carries version N as the basis. The server applies it anyway and the record becomes what User B typed, with no trace of what User A wrote. From the database's perspective nothing went wrong: it received a valid write and stored it. But from the product's perspective, data was silently destroyed.

This is the classic Lost Update anomaly and it occurs any time two transactions overlap on the same record without isolation. The fix requires making writes conditional on a version match so that the second writer is forced to acknowledge the conflict before proceeding.

Problem 2: Dropped Webhooks (Coordination)

Clerk uses an event-driven model. When a subscription is cancelled it fires a webhook POST to the backend. The backend's current handler processes the event once and does not record that it was processed. This creates two compounding vulnerabilities.

First, if the network drops the request or the server returns a 5xx before writing to the database, the cancellation is silently lost. The user's record stays as premium and no retry mechanism exists on the backend side to recover. Clerk does retry failed deliveries, but if those retries also arrive during downtime they too will be dropped.

Second, even if the event is eventually delivered, there is no idempotency guard. Clerk can legitimately deliver the same event more than once as a reliability measure, and without a

deduplication mechanism the handler will apply the cancellation action multiple times. In this case double-applying a downgrade is harmless, but in other webhooks (billing credits, provisioning) it would cause real damage. The fundamental issue is that the backend treats webhooks as fire-and-forget rather than as durable, at-least-once messages that need acknowledgement and deduplication.

Problem 3: Synchronous LLM Call (Fault Tolerance)

The LLM endpoint is invoked synchronously inside a request handler. FastAPI is built on asyncio, so an async handler that awaits a slow I/O call should theoretically not block other requests. However, the problem is subtler: the application has a bounded thread pool and connection pool, and every concurrent request that is suspended waiting on the LLM still occupies a slot in that pool.

When the external LLM API goes down, it does not fail immediately. Instead it hangs for its full 60-second TCP timeout before raising an exception. During those 60 seconds the suspended coroutine holds resources. With enough users hitting the endpoint, all available worker slots fill up with suspended coroutines waiting on a dead service. New requests for any endpoint, not just the LLM one, start queuing behind them. To outside observers the entire application appears to be down. This is a cascading failure caused by a single dependency becoming a blocking single point of failure with no timeout, no retry budget, and no fallback path.

Part 2: Architectural Fixes and Design

Fix 1: Optimistic Locking for Shared Documents

The cleanest fix here is optimistic locking with monotonic version numbers. Every document record stores an integer version field alongside its content. The version is included in the GET response that clients read. When a client submits an update it must include the version it based its edit on. The server compares the submitted version against the current stored version before writing.

If the versions match, the write proceeds and the version is atomically incremented. If they do not match, the server returns a 409 Conflict and the client must fetch the latest state, apply a merge or show the user a conflict warning, and retry. No data is silently overwritten. The key insight is that the version check and write happen in a single atomic database operation (a conditional UPDATE WHERE version = X), which means two concurrent writers cannot both win.

This is called optimistic locking because it assumes conflicts are rare and does not hold a database lock for the entire edit session. It only validates at commit time. An alternative would be pessimistic locking, where the first user to open a document acquires a lock and others are blocked until that lock is released. Pessimistic locking gives stronger guarantees but is far worse for user experience since any network interruption can strand the lock indefinitely.

UML Sequence Diagram: Concurrent Edit with Optimistic Locking

Actor	Message / Action	Target	Outcome
User A	GET /documents/doc-1	**Server**	Returns {content, version: 1}
User B	GET /documents/doc-1	**Server**	Returns {content, version: 1}
User A	PUT /documents/doc-1 {version:1, content:"A edits"}	**Server**	200 OK — saved as version 2
User B	PUT /documents/doc-1 {version:1, content:"B edits"}	**Server**	409 Conflict — version mismatch
User B	GET /documents/doc-1 (refresh)	**Server**	Returns {content: "A edits", version: 2}
User B	PUT /documents/doc-1 {version:2, content:"merged"}	**Server**	200 OK — saved as version 3

Fix 2: Fault-Tolerant Webhook Handler

The design has two components: idempotency keys and a dead-letter queue.

Every Clerk webhook delivery includes a unique event ID in the svix-id header. The backend stores each processed event ID in a dedicated table. Before processing any webhook, the handler checks this table. If the ID is already there, it returns 200 immediately and does nothing. If it is new, it processes the event and then writes the ID to the table as part of the same database transaction. This means even if Clerk retries the same event a hundred times, the cancellation is applied exactly once.

For the dropped-delivery problem, a dead-letter queue (DLQ) is added. If the handler fails after receiving the event (for instance, the database write fails), it writes the raw event payload to the DLQ. A separate background worker polls the DLQ and replays events using the same idempotent handler. Because the handler is idempotent, replaying is always safe. The combination means that as long as the event reaches the backend at least once and the DLQ persists, it will eventually be processed exactly once.

Fix 3: Circuit Breaker with Fallback

A circuit breaker is a state machine with three states: Closed, Open, and Half-Open.

- Closed: Requests flow through normally. The breaker counts consecutive failures.
- Open: After the failure threshold is crossed, the breaker trips. All requests return the fallback immediately without touching the external service. A timer starts.
- Half-Open: After the recovery timeout expires, one probe request is allowed through. If it succeeds, the breaker closes. If it fails, the breaker re-opens and the timer resets.

For the LLM endpoint, every call is wrapped with asyncio.wait_for and a short timeout (e.g., 5 seconds). This prevents the 60-second hang entirely. The fallback response is a user-friendly message explaining that the AI tutor is temporarily unavailable. From the user's perspective they get an instant, graceful response instead of a hung browser tab. From the server's perspective no resources are held waiting on a dead dependency. Once the LLM service recovers, the Half-Open probe detects it and normal operation resumes automatically.

CAP Theorem Trade-offs

These fixes each make a deliberate trade-off in the consistency-availability-latency space that is worth naming explicitly.

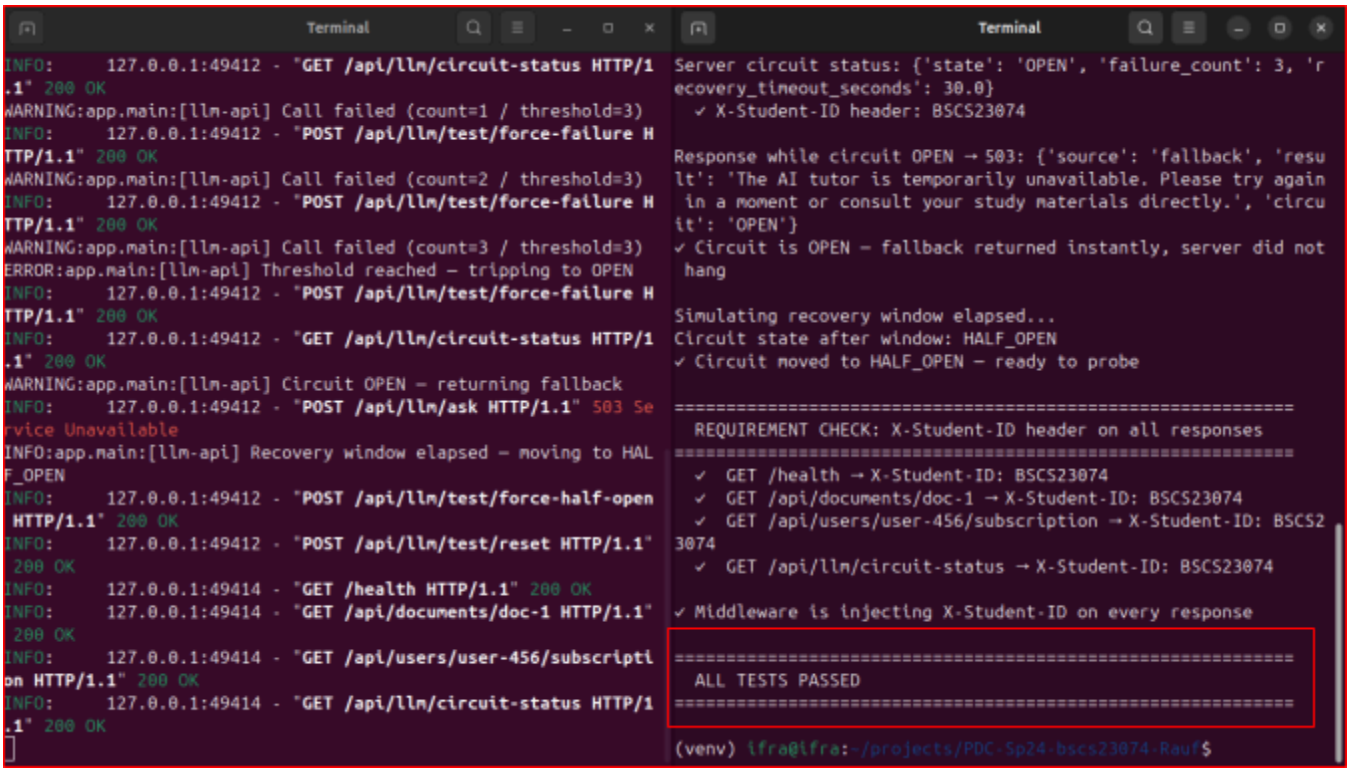
Optimistic locking prioritizes consistency over availability. When a version conflict occurs, the second writer's request is rejected. The user must do extra work (refresh, merge, retry) before their write succeeds. The system refuses to serve a stale write, which is the right call for a shared document editor where data integrity matters more than write throughput. We are in the CP quadrant for concurrent writes.

The idempotent webhook handler also prioritizes consistency. We would rather process an event once correctly than risk processing it zero times or twice. The DLQ adds a form of eventual consistency: the system may be briefly out of sync if the DLQ worker has a backlog, but it will always converge to the correct state. This is a reasonable trade-off because a subscription state error (user appearing premium after cancellation) is a business-critical inconsistency.

The circuit breaker prioritizes availability over consistency. When the LLM is down, we return a fallback rather than an accurate response. Strictly speaking, a fallback is not the same as a real AI answer, so we are sacrificing correctness for liveness. This is the correct call here because the AI feature is supplementary: it is better to keep the whole application running with degraded AI functionality than to let one broken dependency take everything down. We are explicitly choosing AP for the LLM feature.

The broader lesson is that there is no globally correct answer. Each sub-system demands its own trade-off analysis based on what failure mode is more acceptable to the business.

Code run test:



Left side is server. Right side is a file to test it.

As seen in SS, all tests are passed successfully. Github link is pasted at top with instructions in README to test and run the code.